NESNE YÖNELİMLİ PROGRAMLAMA - 1

Özel (Magic) Metotlar

ve operatör aşırı yükleme: nesnelerinize Python'un dilini öğretin

`__str__` `__eq__` `__add__` `__len__`

vektor.py

```python
v1 = Vektor(2, 3)
v2 = Vektor(5, -1)
print(v1 + v2)

# Vektor(7, 2)  ← ders sonunda
```

Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

Gösterim

Nesnenin print ve konsolda nasıl görüneceğini belirlemek

Karşılaştırma

==, < ve sıralamayı kendi sınıfınıza öğretmek

Aritmetik

+, *, += gibi operatörleri aşırı yüklemek

Kapsayıcı

len, [ ], in ve for ile çalışan sınıflar yazmak

Renk kodu: yeşil = gösterim, turuncu = karşılaştırma, mor = aritmetik, mavi = kapsayıcı.

GEÇEN HAFTADAN BUGÜNE

Nesne korunuyor, ama konuşamıyor

sorun.py

```
class BankaHesabi:
    def __init__(self, sahip, bakiye):
        self.sahip = sahip
        self.bakiye = bakiye
h1 = BankaHesabi("Erhan", 500)
h2 = BankaHesabi("Erhan", 500)
print(h1)
print(h1 == h2)
```

▷ ÇIKTI

```
<__main__.BankaHesabi object at 0x7f1e46adecf0>
False
```

- ⚠ **print(h1)** anlamsız bir bellek adresi yazıyor.
- ⚠ Aynı bilgiye sahip iki hesap **eşit değil**.
- ⚠ **h1 + h2**, **len(h1)** hiç çalışmaz.

Bugünün sorusu

Kendi sınıfımızın nesneleri **print**, **==**, **+** ve **len** ile nasıl anlamlı çalışır?

KAVRAM

Her sözdiziminin arkasında bir metot var

```
print(len("merhaba"), "merhaba".__len__())
print(3 + 4, (3).__add__(4))
print([1, 2] == [1, 2], [1, 2].__eq__([1, 2]))

sihirli = [m for m in dir(int) if m.startswith("__")]
print(len(sihirli))
```

▷ ÇIKTI

```
7 7
7 7
True True
63
```

Dunder = double underscore

Adı iki alt çizgiyle başlayıp biten metotlar: **__init__**, **__len__**, **__add__**... Türkçede **özel metot** ya da **magic metot** denir.

Biz yazarız, Python çağırır

len(x) yazdığınızda Python arka planda **x.__len__()** çağırır. Bu metotları kendimiz doğrudan çağırmayız.

int sınıfında tam **63** dunder var. Bugün en önemlilerini kendi sınıflarımıza yazacağız.

HARİTA

Siz yazarsınız, Python çağırır

Siz yazarsınız	Python çağırır	Bölüm
print(x)	x.__str__()	Gösterim
repr(x)	x.__repr__()	Gösterim
a == b	a.__eq__(b)	Karşılaştırma
a < b	a.__lt__(b)	Karşılaştırma
a + b	a.__add__(b)	Aritmetik
a * 3	a.__mul__(3)	Aritmetik
len(x)	x.__len__()	Kapsayıcı
x[i]	x.__getitem__(i)	Kapsayıcı
e in x	x.__contains__(e)	Kapsayıcı

BÖLÜM 1

01

__str__ & __repr__

Nesneniz kendini nasıl tanıtır?

print() str() repr() f-string liste içinde

Varsayılan gösterim: kimlik kartı yok

```
class Kitap:
    def __init__(self, ad, yazar):
        self.ad = ad
        self.yazar = yazar

k = Kitap("Kürk Mantolu Madonna",
          "Sabahattin Ali")
print(k)
```

▷ ÇIKTI

```
<__main__.Kitap object at 0x7f1e46adf620>
```

Çıktının anatomisi

- `__main__` — sınıfın tanımlandığı modül
- `Kitap` — sınıfın adı
- `0x7f1e…` — bellekteki adres

Eksik olan

Kitabın **adı** ve **yazarı**. Python bunları nasıl göstereceğimizi bilemez; ona biz söylemeliyiz.

__str__: kullanıcı için okunur metin

```python
class Kitap:
    def __init__(self, ad, yazar):
        self.ad = ad
        self.yazar = yazar

    def __str__(self):
        return f"{self.ad} - {self.yazar}"

k = Kitap("Kürk Mantolu Madonna",
          "Sabahattin Ali")
print(k)
s = str(k)
print(len(s))
print(f"Okuyorum: {k}")
```

▷ ÇIKTI

```
Kürk Mantolu Madonna - Sabahattin Ali
37
Okuyorum: Kürk Mantolu Madonna - Sabahattin Ali
```

Ne zaman çağrılır?

print(k), **str(k)** ve **f"{k}"** yazıldığında.

⚠ Mutlaka bir **str döndürmeli**. print etmek değil, **return** etmek gerekir.

__repr__: geliştirici için kesin tarif

```python
class Kitap:
    def __init__(self, ad, yazar):
        self.ad = ad
        self.yazar = yazar
    def __str__(self):
        return f"{self.ad} - {self.yazar}"
    def __repr__(self):
        return f"Kitap({self.ad!r}, {self.yazar!r})"

k = Kitap("Sefiller", "Victor Hugo")
print(str(k))
print(repr(k))
raf = [k, Kitap("Nutuk", "Atatürk")]
print(raf)
print(f"{k!r}")
```

▷ ÇIKTI

```
Sefiller - Victor Hugo
Kitap('Sefiller', 'Victor Hugo')
[Kitap('Sefiller', 'Victor Hugo'),
 Kitap('Nutuk', 'Atatürk')]
Kitap('Sefiller', 'Victor Hugo')
```

Altın kural

repr çıktısı, mümkünse nesneyi yeniden oluşturan **kod gibi** görünmeli.

Liste yazdırılırken elemanlar **__repr__** ile gösterilir, __str__ ile değil!

KARŞILAŞTIRMA

__str__ mi, __repr__ mi?

	__str__	__repr__
Kime?	Son kullanıcıya	Geliştiriciye
Amaç	Okunur, sade	Kesin, belirsizliksiz
Çağıranlar	print(x), str(x), f"{x}"	repr(x), f"{x!r}", konsol, liste içi
Örnek	Sefiller – Victor Hugo	Kitap('Sefiller', 'Victor Hugo')
Yazılmazsa	__repr__ kullanılır	<... object at 0x...>

☆ **Tek birini yazacaksanız: __repr__.** Çünkü __str__ yoksa Python ona düşer; tersi geçerli değildir.

GÖSTERİM · YEDEK

__str__ yoksa Python __repr__'a düşer

```python
class Nokta:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return f"Nokta({self.x}, {self.y})"

p = Nokta(3, 4)
print(p)
print(str(p))
print([p])
```

▷ ÇIKTI

```
Nokta(3, 4)
Nokta(3, 4)
[Nokta(3, 4)]
```

Sonuç

Tek bir **__repr__**, üç farklı kullanımın hepsini güzelleştirdi.

print(p)
yazıldı

→

__str__ ?
sınıfta yok

→

__repr__ ?
var, onu kullan

→

Nokta(3, 4)
ekrana yazıldı

BÖLÜM 2

02

__eq__ & __lt__

Nesneler ne zaman eşit, hangisi küçük?

`==` `!=` `<` `sorted()` `@total_ordering` `__hash__`

KARŞILAŞTIRMA · SORUN

Varsayılan ==, kimliğe bakar

```
class Para:
    def __init__(self, miktar, birim):
        self.miktar = miktar
        self.birim = birim

a = Para(100, "TL")
b = Para(100, "TL")
c = a
print(a == b, a is b, a == c)
```

▷ ÇIKTI `False False True`

Bellekte ne var?

a → `Para(100, TL) · nesne 1`

c → `Para(100, TL) · nesne 1`

b → `Para(100, TL) · nesne 2`

__eq__ yazılmadıysa **a == b**, **a is b** ile aynı şeyi sorar: “ikisi aynı nesne mi?”

__eq__: eşitliği biz tanımlarız

```
class Para:
    def __init__(self, miktar, birim):
        self.miktar = miktar
        self.birim = birim

    def __eq__(self, other):
        if not isinstance(other, Para):
            return NotImplemented
        return (self.miktar == other.miktar
                and self.birim == other.birim)

a = Para(100, "TL"); b = Para(100, "TL")
d = Para(100, "USD")
print(a == b, a == d, a != d, a is b)
print(a == 100)
```

▷ ÇIKTI

```
True False True False
False
```

!= bedava

__eq__ yazınca Python **!=** için sonucu otomatik olarak tersine çevirir.

a is b hâlâ False: iki ayrı nesne, ama artık eşitler.

KARŞILAŞTIRMA · DERİNLEMESİNE

NotImplemented: “bilmiyorum, sen dene”

a == 100 nasıl False oldu?

1. Python **a.__eq__(100)** çağırır → **NotImplemented**
2. Karşı tarafa sorar: **(100).__eq__(a)** → **NotImplemented**
3. İkisi de bilmiyor → kimlik karşılaştırması yapılır
4. **a is 100** değil → sonuç **False**

KONTROLSÜZ YAZILIRSA

```
def __eq__(self, other):
    return self.miktar == other.miktar

Para(1, "TL") == 100
```

▷ ÇIKTI

```
Hata: 'int' object has no attribute 'miktar'
```

KARŞILAŞTIRMA · __lt__

__lt__ ile sıralama kendiliğinden gelir

```
class Ogrenci:
    def __init__(self, ad, ort):
        self.ad = ad
        self.ort = ort
    def __lt__(self, other):
        return self.ort < other.ort
    def __repr__(self):
        return f"{self.ad}({self.ort})"

s = [Ogrenci("Ali", 72), Ogrenci("Can", 91),
     Ogrenci("Ece", 85)]
print(sorted(s))
print(max(s))
print(s[0] < s[1])
print(s[0] <= s[1])
```

▷ ÇIKTI

```
[Ali(72), Ece(85), Can(91)]
Can(91)
True
TypeError: '<=' not supported
between instances of 'Ogrenci'
and 'Ogrenci'
```

Tek metot, üç kazanç

sorted(), **min()** ve **max()** karşılaştırmak için **<** kullanır.

⚠ **<=** ayrı bir metottur (**__le__**) ve henüz yok.

KARŞILAŞTIRMA · KISAYOL

İkisini yaz, gerisini Python türetsin

```python
from functools import total_ordering

@total_ordering
class Ogrenci:
    def __init__(self, ad, ort):
        self.ad = ad
        self.ort = ort
    def __eq__(self, other):
        return self.ort == other.ort
    def __lt__(self, other):
        return self.ort < other.ort

a = Ogrenci("Ali", 72)
b = Ogrenci("Can", 91)
print(a <= b, a >= b, a > b, a != b)
```

▷ **ÇIKTI** `True False False True`

Siz yazdınız	Dekoratör ekledi
__eq__	__le__ (<=)
__lt__	__gt__ (>)
	__ge__ (>=)

@total_ordering, **functools** modülündendir; import etmeyi unutmayın.

KARŞILAŞTIRMA · DİKKAT

__eq__ yazınca __hash__ kaybolur

```
class Para:
    def __init__(self, miktar, birim):
        self.miktar = miktar
        self.birim = birim
    def __eq__(self, other):
        return ((self.miktar, self.birim)
                == (other.miktar, other.birim))

try: print({Para(5, "TL")})
except TypeError as e: print("Hata:", e)

class Para2(Para):
    def __hash__(self):
        return hash((self.miktar, self.birim))

print(len({Para2(5, "TL"), Para2(5, "TL"),
           Para2(7, "TL")}))
```

▷ ÇIKTI

```
Hata: unhashable type: 'Para'
2
```

Neden?

Küme ve sözlük anahtarı **hash** ister. **__eq__** tanımlanınca Python eski hash'i kapatır: **Para.__hash__** artık **None**.

Kural

Eşit nesnelerin hash'i de **eşit olmalı**. Bu yüzden aynı nitelikleri hash'leriz.

BÖLÜM 3

03

__add__ & arkadaşları

Operatör aşırı yükleme: + işaretine yeni anlam

+ - * __radd__ += -x, abs()

KAVRAM

Aynı +, farklı anlamlar

`3 + 4`	`"ab" + "cd"`	`[1] + [2]`	`v1 + v2`
↓	↓	↓	↓
`7`	`"abcd"`	`[1, 2]`	`Vektor(7, 2)`
int: toplama	str: birleştirme	list: uç uca ekleme	bizim sınıfımız

Operatör aşırı yükleme (operator overloading)

Var olan bir operatörün **kendi sınıfımız için** ne yapacağını tanımlamak. Her tür + işaretine kendi anlamını verir; Python hangi **__add__**'in çalışacağına **soldaki nesnenin türüne** bakarak karar verir.

▷ **ÇIKTI**
```
7 abcd [1, 2]
```

ARİTMETİK · __add__

Vektör toplama

```
class Vektor:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        return Vektor(self.x + other.x,
                      self.y + other.y)
    def __sub__(self, other):
        return Vektor(self.x - other.x,
                      self.y - other.y)
    def __repr__(self):
        return f"Vektor({self.x}, {self.y})"

v1 = Vektor(2, 3); v2 = Vektor(5, -1)
print(v1 + v2); print(v1 - v2)
print(v1.__add__(v2))
print(v1 + v2 + Vektor(1, 1)); print(v1)
```

▷ ÇIKTI

```
Vektor(7, 2)
Vektor(-3, 4)
Vektor(7, 2)
Vektor(8, 3)
Vektor(2, 3)
```

Yeni nesne döndür

__add__ sonucu **yeni** bir Vektor olarak döndürür. v1 değişmedi: son satır hâlâ Vektor(2, 3).

Zincir çalışır: (v1 + v2) bir Vektor, ona + Vektor(1, 1) eklenir.

ARİTMETİK · ADIM ADIM

v1 + v2 yazınca ne olur?

1 · SİZ
`v1 + v2`

→

2 · PYTHON
`v1.__add__(v2)`

→

3 · ATAMA
`self = v1`
`other = v2`

→

4 · HESAP
`Vektor(2+5, 3+(-1))`

→

5 · SONUÇ
`Vektor(7, 2)`

Sol taraf belirler

Python **soldaki** nesnenin metodunu çağırır. **v1 + v2** için Vektor'ün __add__'i, **3 + v1** için int'in __add__'i çalışır.

self ve other

self operatörün solundaki, **other** sağındaki nesnedir. İsim zorunlu değil ama gelenek böyle.

Dönüş değeri

Ne döndürürseniz ifadenin sonucu odur. **return** unutulursa sonuç **None** olur.

ARİTMETİK · __mul__

v * 3 çalışıyor, 3 * v neden çalışmıyor?

```
class Vektor:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __mul__(self, k):
        return Vektor(self.x * k, self.y * k)
    def __repr__(self):
        return f"Vektor({self.x}, {self.y})"

v = Vektor(2, 3)
print(v * 3)
print(3 * v)
```

▷ ÇIKTI

```
Vektor(6, 9)
TypeError: unsupported operand type(s)
for *: 'int' and 'Vektor'
```

3 * v için Python'un denediği

1. **(3).__mul__(v)** → int, Vektor'ü tanımaz → NotImplemented
2. **v.__rmul__(3)** → böyle bir metot yok
3. Kimse yapamadı → **TypeError**

Çözüm: **sağdan** çalışan sürüm, **__rmul__**.

__rmul__ ve __radd__: sağdayken de çalış

```python
class Vektor:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, o):
        return Vektor(self.x + o.x, self.y + o.y)
    def __mul__(self, k):
        return Vektor(self.x * k, self.y * k)
    def __rmul__(self, k):
        return self * k
    def __radd__(self, o):
        if o == 0:
            return self
        return self + o
    def __repr__(self):
        return f"Vektor({self.x}, {self.y})"
v = Vektor(2, 3); print(3 * v)
print(sum([Vektor(1, 2), Vektor(3, 4), Vektor(5, 6)]))
```

▷ ÇIKTI

```
Vektor(6, 9)
Vektor(9, 12)
```

__rmul__

3 * v → int vazgeçince Python **v.__rmul__(3)** çağırır. Biz de işi **__mul__**'e devrettik.

__radd__ ve sum()

sum() toplama **0**'dan başlar: ilk adım **0 + Vektor**. __radd__ olmadan: unsupported operand type(s) for +: 'int' and ...

ARİTMETİK · YERİNDE İŞLEM

+= için __iadd__

```python
class Sepet:
    def __init__(self):
        self.urunler = []
    def __iadd__(self, urun):
        self.urunler.append(urun)
        return self          # şart!
    def __repr__(self):
        return f"Sepet({self.urunler})"

s = Sepet(); kimlik = id(s)
s += "elma"; s += "armut"
print(s, id(s) == kimlik)

# Vektor: önceki slayttaki sınıf
v = Vektor(1, 1); kimlik = id(v)
v += Vektor(1, 1)
print(v, id(v) == kimlik)
```

▷ ÇIKTI

```
Sepet(['elma', 'armut']) True
Vektor(2, 2) False
```

İki yol

__iadd__ varsa nesne **yerinde** değişir (aynı id). Yoksa Python **v = v + ...** yapar: **__add__** ile **yeni** nesne.

⚠ **return self** unutulursa **s** değişkeni **None** olur!

BAŞVURU

Operatör → metot tablosu

İşlem	Normal	Sağdan	Yerinde
a + b	__add__	__radd__	__iadd__
a - b	__sub__	__rsub__	__isub__
a * b	__mul__	__rmul__	__imul__
a / b	__truediv__	__rtruediv__	__itruediv__
a // b	__floordiv__	__rfloordiv__	__ifloordiv__
a % b	__mod__	__rmod__	__imod__
a ** b	__pow__	__rpow__	__ipow__

```
v = Vektor(3, 4)
print(-v, abs(v))
```

▷ ÇIKTI `Vektor(-3, -4) 5.0`

Tek operandlı: **-v** → **__neg__**, **abs(v)** → **__abs__**

BÖLÜM 4

04

__len__, __getitem__ …

Nesneniz bir koleksiyon gibi davransın

len() x[i] in for bool() x()

Kendi listemiz: çalma listesi

```
class Calma:
    def __init__(self, *sarkilar):
        self._s = list(sarkilar)
    def __len__(self):
        return len(self._s)
    def __getitem__(self, i):
        return self._s[i]

c = Calma("Bir Derdim Var", "Gesi Bağları",
          "Islak Islak", "Yalnızlık Senfonisi")
print(len(c))
print(c[0]); print(c[-1]); print(c[1:3])
for s in c:
    print("♪", s)
print("Gesi Bağları" in c)
```

▷ ÇIKTI

```
4
Bir Derdim Var
Yalnızlık Senfonisi
['Gesi Bağları', 'Islak Islak']
♪ Bir Derdim Var
♪ Gesi Bağları
♪ Islak Islak
♪ Yalnızlık Senfonisi
True
```

__getitem__ tek başına **dilimleme**, **for** ve **in**'i de çalıştırdı.

in ve for'u kendimiz tanımlayalım

```python
class Takvim:
    def __init__(self):
        self._gun = {}
    def ekle(self, tarih, olay):
        self._gun[tarih] = olay
    def __contains__(self, tarih):
        return tarih in self._gun
    def __iter__(self):
        return iter(sorted(self._gun))

t = Takvim()
t.ekle("2026-11-09", "Vize")
t.ekle("2026-10-29", "Bayram")
print("2026-11-09" in t, "2026-12-01" in t)
for g in t:
    print(g)
```

▷ ÇIKTI

```
True False
2026-10-29
2026-11-09
```

__contains__

x in t → t.__contains__(x). True/False döndürür; sözlükte arama çok hızlıdır.

__iter__

for döngüsüne bir **iterator** verir. Burada tarihleri **sıralı** gezdik; ekleme sırası önemli değil.

Nesne ne zaman “doğru” sayılır?

```
class Sepet:
    def __init__(self):
        self.urunler = []
    def __len__(self):
        return len(self.urunler)

s = Sepet()
print(bool(s))
s.urunler.append("elma")
print(bool(s))
if s:
    print("Sepet dolu")
```

▷ ÇIKTI

```
False
True
Sepet dolu
```

```
class Hesap:
    def __init__(self, b):
        self.b = b
    def __bool__(self):
        return self.b > 0

print(bool(Hesap(0)), bool(Hesap(50)))
```

▷ ÇIKTI `False True`

Python’un sırası

if x: önce **__bool__**’a bakar, yoksa **__len__**’e (0 ise False), o da yoksa her nesne **True**’dur.

KAPSAYICI · __call__

Fonksiyon gibi çağrılabilen nesne

```
class Indirim:
    def __init__(self, oran):
        self.oran = oran
    def __call__(self, fiyat):
        return fiyat * (1 - self.oran)

kara_cuma = Indirim(0.25)
print(kara_cuma(400))
print(callable(kara_cuma))
print(list(map(kara_cuma, [100, 80])))
```

▷ ÇIKTI

```
300.0
True
[75.0, 60.0]
```

Durum + davranış

Fonksiyon gibi çağrılır ama **oran** bilgisini nesne içinde saklar. Farklı oranlarla farklı indirim nesneleri üretebiliriz.

map() gibi fonksiyon bekleyen her yere verilebilir.

BÜYÜK RESİM

Bugün öğrendiğimiz tüm metotlar

Gösterim

`__str__`

print, str, f-string

`__repr__`

repr, konsol, liste içi

Karşılaştırma

`__eq__`

== ve !=

`__lt__`

<, sorted, max

`__hash__`

küme, sözlük anahtarı

Aritmetik

`__add__`

+ (ve -, *, /)

`__radd__`

sağdan +, sum()

`__iadd__`

+= yerinde

`__neg__`

-x

Kapsayıcı

`__len__`

len()

`__getitem__`

x[i], dilim

`__contains__`

in

`__iter__`

for

`__bool__`

if x:

`__call__`

x()

Para sınıfı: dört bölüm tek sınıfta

```python
from functools import total_ordering

@total_ordering
class Para:
    def __init__(self, miktar, birim="TL"):
        self.miktar = round(miktar, 2)
        self.birim = birim
    def __repr__(self):
        return f"Para({self.miktar}, {self.birim!r})"
    def __str__(self):
        return f"{self.miktar:,.2f} {self.birim}"
    def __eq__(self, o):
        if not isinstance(o, Para):
            return NotImplemented
        return ((self.miktar, self.birim)
                == (o.miktar, o.birim))
    def __hash__(self):
        return hash((self.miktar, self.birim))
```

```python
    def __lt__(self, o):
        self.__ayni_birim(o)
        return self.miktar < o.miktar
    def __add__(self, o):
        self.__ayni_birim(o)
        return Para(self.miktar + o.miktar, self.birim)
    def __radd__(self, o):
        return self if o == 0 else self + o
    def __mul__(self, k):
        return Para(self.miktar * k, self.birim)
    __rmul__ = __mul__
    def __bool__(self):
        return self.miktar != 0
    def __ayni_birim(self, o):
        if self.birim != o.birim:
            raise ValueError("Birimler farklı")
```

UYGULAMA · KULLANIM

Para sınıfını kullanalım

```
kalem = Para(15.5); defter = Para(42)
sepet = [kalem, defter, 3 * kalem]
toplam = sum(sepet)
print(toplam)
print(repr(toplam))
print(max(sepet), kalem <= defter)
print(sorted(sepet))
print(Para(0) or "Bedava")
print(len({Para(5), Para(5.0), Para(7)}))
try:
    Para(10) + Para(10, "USD")
except ValueError as e:
    print("Hata:", e)
```

▷ ÇIKTI

```
104.00 TL
Para(104.0, 'TL')
46.50 TL True
[Para(15.5, 'TL'), Para(42, 'TL'),
 Para(46.5, 'TL')]
Bedava
2
Hata: Birimler farklı
```

İfade	Çalışan metot
3 * kalem	__rmul__
sum(sepet)	__radd__, __add__
kalem <= defter	__le__ (türetilen)
Para(0) or …	__bool__

SÖZLÜK

Bu haftanın terimleri

Terim	İngilizce	Anlamı
Özel metot	magic / dunder method	Python'un belirli işlemlerde kendiliğinden çağırdığı __ad__ metotları
Operatör aşırı yükleme	operator overloading	Bir operatöre kendi sınıfımız için anlam vermek
Metin gösterimi	__str__ / __repr__	Kullanıcıya ve geliştiriciye yönelik metin
Yansıtılmış metot	reflected (r) method	Nesne operatörün sağındayken çağrılan __radd__ vb.
Yerinde işlem	in-place (i) method	+= gibi, nesneyi değiştiren __iadd__ vb.
NotImplemented	NotImplemented	"Bu işlemi bilmiyorum" anlamında dönen özel değer
Özet değer	hash	Küme/sözlükte hızlı arama için tamsayı
Çağrılabilir	callable	() ile çağrılabilen nesne (__call__)

KONTROL LİSTESİ

Magic metotları doğru yazdım mı?

- Her sınıfta en azından bir **__repr__** var mı?
- **__str__** ve **__repr__** metin **döndürüyor** mu (print değil)?
- **__eq__** yabancı türde **NotImplemented** döndürüyor mu?
- Küme/sözlükte kullanılacaksa **__hash__** yazıldı mı?
- Aritmetik metotlar **yeni nesne** döndürüyor mu?
- **3 * x**, **sum()** için **r**-sürümler var mı?
- **__iadd__** sonunda **return self** var mı?
- Operatörün anlamı **sezgisel** mi? (iki hesap toplanmaz)

ÇIKTIYI TAHMİN ET

Ekrana ne yazılır?

SORU 1

```
class A:
    def __repr__(self):
        return "R"
    def __str__(self):
        return "S"

a = A()
print(a, [a], f"{a!r}")
```

S [R] R: liste içi ve !r, repr kullanır.

SORU 2

```
class S:
    def __init__(self, n):
        self.n = n
    def __add__(self, o):
        return S(self.n + o.n)
    def __eq__(self, o):
        return self.n == o.n

print(S(1) + S(2) == S(3),
      S(1) + S(2) is S(3))
```

True False: eşit ama aynı nesne değil.

DİKKAT

Sık yapılan altı hata

`__str__` içinde print

Metot None döndürür: **__str__ returned non-string (type NoneType)**

`__eq__`'de tür kontrolü yok

a == 100 → **'int' object has no attribute 'miktar'**. NotImplemented döndürün.

`__hash__`'i unutmak

__eq__ yazınca kümeye konamaz: **unhashable type**.

`__add__` içinde self'i değiştirmek

a + b sonrası **a** da değişir. Her zaman yeni nesne döndürün.

`__iadd__`'de return yok

s += x sonrası **s** değişkeni **None** olur.

`r-sürümü unutmak`

3 * v ve **sum()** TypeError verir: **unsupported operand type(s)**.

ÖZET

Bu hafta neler öğrendik?

Gösterim

__repr__ geliştirici için, __str__ kullanıcı için. Önce __repr__.

Karşılaştırma

__eq__ + __lt__ ve @total_ordering. __eq__ varsa __hash__'i düşün.

Aritmetik

Yeni nesne döndür; sağdan işlem için r-, += için i-sürümleri.

Kapsayıcı

__len__, __getitem__, __contains__, __iter__, __bool__, __call__.

Magic metotlar, sınıfınızı Python'un **yerleşik türleri kadar doğal** kullanılır hâle getirir.

ÖDEV

Haftaya kadar

1 • Kesir

Kesir(3, 4): __str__ “3/4”, __repr__, __eq__ (sadeleştirerek; math.gcd), __add__, __mul__.

2 • Süre

Sure(dakika): __str__ “1 sa 25 dk”, __add__, __lt__ + @total_ordering, sum() için __radd__.

3 • Para’yı genişlet

__sub__ ekleyin; sonuç negatifse ValueError. Para(10) * 1.5 ve 2 * Para(10) test edin.

4 • Oynatma listesi

__len__, __getitem__, __contains__ ve şarkı ekleyen __iadd__ (return self!).

GELECEK HAFTA · 7

Uygulamalı tekrar

1–6. haftalar tek projede: sınıf, nesne, kapsülleme, @property ve magic metotlar birlikte.

Hazırlık: 3–6. haftaların ödevlerini getirin.